FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

PROJET DE FIN DE MODULE

DEVOPS & CLOUD COMPUTING

Master Excellence en Intelligence Artificielle

Sous le thème :

Infrastructure DevOps Automatisée sur Oracle Cloud : CI/CD, Kubernetes et Monitoring

Réalisé par :

Hajar ELKHALIDI & Nohaila ICHOU

Encadré par :

Prof. Faouzia BENABBOU

Année universitaire : 2025–2026

Table des matières

Résumé **5**

Abstract **6**

1 Introduction générale **7**
1.1 Contexte du projet 7
1.2 Organisme d'accueil 7
1.3 Problématique et besoins 7
1.4 Objectifs 8
1.5 Structure du rapport 8

2 Étude de l'existant et choix technologiques **9**
2.1 Panorama des outils DevOps 9
2.1.1 Plateforme cloud 9
2.1.2 Infrastructure as Code 9
2.1.3 Conteneurisation et orchestration 9
2.1.4 CI/CD 10
2.1.5 Monitoring 10
2.2 Architecture technique retenue 10

3 Mise en place de l'infrastructure **11**
3.1 Préparation de l'environnement local 11
3.1.1 Installation de Terraform 11
3.2 Création manuelle des VMs sur Oracle Cloud 12
3.2.1 Informations de base et placement 12
3.2.2 Choix de l'image et du shape 12
3.2.3 Configuration réseau 13
3.2.4 Injection de la clé SSH 14
3.2.5 Récapitulatif et création 14
3.2.6 Configuration des accès Oracle Cloud 15
3.2.7 Création du compartiment Oracle Cloud 16
3.2.8 Configuration des Policies IAM 17
3.2.9 Configuration des règles réseau (Security List) 18
3.3 Provisionnement avec Terraform 19
3.3.1 Résolution du problème réseau 19

3.3.2 Description de l'infrastructure déclarée 19
3.3.3 Exécution et résultats 20
3.4 Configuration avec Ansible 20
3.4.1 Inventaire et connectivité 20
3.4.2 Playbook de configuration 21
3.4.3 Exécution du playbook 22
3.5 Déploiement du cluster Kubernetes 23
3.5.1 Initialisation du nœud master 23
3.5.2 Jonction du nœud worker 24
3.5.3 Vérification de l'état du cluster 24
3.5.4 Problèmes rencontrés et solutions 25

4 Application, CI/CD et Monitoring 26
4.1 Application Flask conteneurisée 26
4.1.1 Code source 26
4.1.2 Dockerfile et build multi-architecture 27
4.1.3 Tests unitaires 27
4.2 Pipeline CI/CD avec GitHub Actions 28
4.2.1 Secrets et prérequis 28
4.2.2 Structure du pipeline 28
4.3 Déploiement Kubernetes de l'application 29
4.3.1 Manifest Deployment 29
4.3.2 Service NodePort 30
4.3.3 Interface utilisateur 30
4.4 Monitoring avec Prometheus et Grafana 31
4.4.1 Déploiement via Helm 31
4.4.2 Composants déployés 32
4.4.3 Configuration et dashboards 32

5 Bilan et conclusion 33
5.1 Synthèse des réalisations 33
5.2 Difficultés rencontrées 33
5.3 Perspectives 34
5.4 Conclusion 34

Table des figures

3.1 Installation des dépendances 11
3.2 Ajout de dépôt Hashicorp en APT 11
3.3 Installation de Terraform sur Ubuntu 24.04 11
3.4 Vérifier la version de Terraform 11
3.5 Informations de base et placement de l'instance k8s-master 12
3.6 Sélection de l'image Ubuntu 22.04 13
3.7 Shape VM.Standard.A1.Flex Always Free 13
3.8 Configuration réseau – sélection du VCN vcn-devops 13
3.9 Injection de la clé SSH publique 14
3.10 Récapitulatif de la configuration avant création 14
3.11 Instances VMs créés 15
3.12 Géneration de cle OCI 15
3.13 Confirguration OCI file preview 16
3.14 Création de compartement 16
3.15 Vérification de compartiment créé 17
3.16 Création des policies IAM pour le compartiment devops 17
3.17 Règles Ingress de la Security List du VCN devops 18
3.18 Vérification de la configuration OCI 18
3.19 Vérification de la configuration OCI 20
3.20 Vérification de la connectivité SSH vers les deux noeuds 22
3.21 Exécution du playbook Ansible sur les noeuds master et worker 22
3.22 Installation du plugin réseau Flannel sur le cluster 23
3.23 Initialisation du master et jonction du worker au cluster 24
3.24 État du cluster Kubernetes après initialisation 24

4.1 Secrets configurés dans le dépôt GitHub Actions 28
4.2 Pipeline CI/CD GitHub Actions exécuté avec succès 29
4.3 Pods de l'application Flask en statut Running 30
4.4 Application Flask accessible dans le navigateur] 30
4.5 Interface web de supervision de l'API Flask 31
4.6 Dashboard Grafana de supervision du cluster Kubernetes] 32

Liste des tableaux

2.1 Spécifications des machines virtuelles Oracle Cloud 10

3.1 Problèmes rencontrés lors du déploiement Kubernetes 25

4.1 Composants du stack de monitoring déployés dans le namespace monitoring . . 32

5.1 Récapitulatif des livrables du projet 33

Résumé

Ce projet a pour objectif la mise en place d'une infrastructure DevOps complète et automatisée sur Oracle Cloud Free Tier, dans le cadre du Master Excellence en Intelligence Artificielle. L'infrastructure repose sur deux machines virtuelles ARM64 configurées en cluster Kubernetes, provisonnées via Terraform et configurées par Ansible.

L'application déployée est une API REST en Python/Flask, conteneurisée avec Docker et publiée sur Docker Hub sous forme d'image multi-architecture (linux/amd64 et linux/arm64). Un pipeline CI/CD basé sur GitHub Actions automatise l'ensemble du cycle de vie applicatif : exécution des tests unitaires, construction de l'image, publication et déploiement sur le cluster Kubernetes. La supervision de l'infrastructure est assurée par Prometheus et Grafana, déployés via Helm.

Ce projet illustre une chaîne DevOps reproductible couvrant l'ensemble des pratiques modernes : Infrastructure as Code, conteneurisation, orchestration, intégration et déploiement continus, et monitoring.

Abstract

This project aims to build a complete and automated DevOps infrastructure on Oracle Cloud Free Tier, as part of the Master of Excellence in Artificial Intelligence program. The infrastructure consists of two ARM64 virtual machines configured as a Kubernetes cluster, provisioned using Terraform and configured with Ansible.

The deployed application is a Python/Flask REST API, containerized with Docker and published to Docker Hub as a multi-architecture image (linux/amd64 and linux/arm64). A GitHub Actions CI/CD pipeline automates the full application lifecycle : unit testing, image building, publishing, and deployment to the Kubernetes cluster. Infrastructure monitoring is handled by Prometheus and Grafana, deployed via Helm.

This project demonstrates a reproducible end-to-end DevOps pipeline covering modern practices : Infrastructure as Code, containerization, orchestration, continuous integration and deployment, and monitoring.

Introduction générale

1.1 Contexte du projet

Le mouvement DevOps représente aujourd'hui un changement de paradigme fondamental dans l'industrie du logiciel. Il s'agit d'une culture et d'un ensemble de pratiques visant à réduire le cloisonnement entre les équipes de développement et d'opérations, afin d'accélérer la livraison de logiciels tout en maintenant un haut niveau de qualité et de stabilité. L'automatisation des infrastructures, la conteneurisation des applications et l'orchestration des déploiements sont au cœur de cette approche.

Ce projet s'inscrit dans cette dynamique en visant la construction d'une infrastructure DevOps complète sur Oracle Cloud Free Tier. Il mobilise un ensemble d'outils représentatifs de l'état de l'art : Terraform pour le provisionnement de l'infrastructure, Ansible pour sa configuration, Docker pour la conteneurisation, Kubernetes pour l'orchestration, GitHub Actions pour l'intégration et le déploiement continus, et enfin Prometheus et Grafana pour le monitoring.

1.2 Organisme d'accueil

Ce projet a été réalisé dans le cadre du Master Excellence en Intelligence Artificielle (MEIA) de la Faculté des Sciences Ben M'Sick (FSBM) de l'Université Hassan II de Casablanca. Le Master MEIA forme des experts capables de développer des solutions innovantes en intelligence artificielle appliquée à divers domaines, incluant l'ingénierie des systèmes et le cloud computing.

1.3 Problématique et besoins

La problématique centrale de ce projet est la suivante : comment mettre en place, de manière automatisée et reproductible, une chaîne DevOps complète sur une infrastructure cloud contrainte (compte Free Tier, architecture ARM64) ? Cette question soulève plusieurs défis techniques : la gestion des restrictions IAM sur Oracle Cloud, la compatibilité des images Docker avec l'architecture ARM, la configuration réseau d'un cluster Kubernetes multi-nœuds, et l'accessibilité distante du cluster depuis un pipeline CI/CD.

1.4 Objectifs

L'objectif général de ce projet est de déployer une application conteneurisée sur un cluster Kubernetes hebergé dans le cloud, en automatisant l'ensemble du processus depuis le code source jusqu'à la production. Les objectifs spécifiques sont :

— Provisionner l'infrastructure réseau sur Oracle Cloud via Terraform

— Configurer les machines virtuelles ARM64 avec Ansible (Docker, Kubernetes)

— Conteneuriser une API REST Python/Flask en image multi-architecture

— Déployer un pipeline CI/CD GitHub Actions (test, build, deploy)

— Orchestrer l'application avec Kubernetes (2 réplicas, sondes de santé, ressources limitées)

— Superviser le cluster avec Prometheus et Grafana

— Produire une documentation détaillée permettant la reproduction de l'infrastructure

1.5 Structure du rapport

Ce rapport est organisé en cinq chapitres. Le chapitre 1 présente le contexte général et les objectifs du projet. Le chapitre 2 expose l'étude de l'existant et les choix technologiques adoptés. Le chapitre 3 décrit la mise en place de l'infrastructure (Terraform, Ansible, Kubernetes). Le chapitre 4 détaille l'application, le pipeline CI/CD et le monitoring. Enfin, le chapitre 5 présente le bilan, les difficultés rencontrées et les perspectives.

Étude de l'existant et choix technologiques

2.1 Panorama des outils DevOps

L'écosystème DevOps est riche d'outils couvrant chaque étape du cycle de vie applicatif. Pour chaque besoin identifié dans ce projet, une évaluation des solutions disponibles a été menée afin de justifier les choix retenus.

2.1.1 Plateforme cloud

Plusieurs fournisseurs cloud proposent des offres gratuites adaptées aux projets académiques : AWS Free Tier, Google Cloud Platform, Microsoft Azure, et Oracle Cloud. Le choix s'est porté sur **Oracle Cloud Free Tier** pour deux raisons principales : l'absence de contrainte bancaire pour l'activation du compte, et la générosité de l'offre Always Free qui inclut quatre vCPUs ARM Ampere A1.Flex et 24 GB de RAM sans limite de temps. La région `af-casablanca-1` disponible réduit par ailleurs la latence réseau depuis le Maroc.

2.1.2 Infrastructure as Code

Terraform s'impose comme la référence pour le provisionnement d'infrastructure multi-cloud. Son approche déclarative (HCL) permet de décrire l'état désiré de l'infrastructure, et son provider OCI officiel assure une compatibilité complète avec Oracle Cloud. Ansible complète Terraform pour la gestion de configuration : là où Terraform crée les ressources, Ansible configure leur système d'exploitation.

2.1.3 Conteneurisation et orchestration

Docker est le standard incontournable de la conteneurisation. Kubernetes s'impose comme la plateforme d'orchestration de référence pour les environnements de production, offrant la mise à l'échelle automatique, la résilience, le déploiement continu et la supervision des conteneurs. L'installation via `kubeadm` a été retenue pour sa proximité avec les déploiements de production.

2.1.4 CI/CD

GitHub Actions a été choisi pour sa intégration native avec GitHub, sa gratuité pour les dépôts publics, et la richesse de son marché d'actions réutilisables. Il permet de définir le pipeline directement dans le dépôt, facilitant sa versioning et sa documentation.

2.1.5 Monitoring

La combinaison Prometheus et Grafana constitue le standard de facto pour la supervision des environnements Kubernetes. Le chart Helm `kube-prometheus-stack` simplifie considérablement leur déploiement en embarquant également Alertmanager et Node Exporter.

2.2 Architecture technique retenue

L'architecture finale du système déployé est présentée dans le tableau ci-dessous. Elle s'articule autour de deux machines virtuelles ARM64 formant un cluster Kubernetes, pilotées depuis une machine locale sous WSL Ubuntu 24.04.

VM	Shape	IP Publique	IP Privée	Rôle
k8s-master	A1.Flex 2 OCPU 12 GB	84.8.221.147	10.0.0.52	Control Plane Kubernetes
k8s-worker	A1.Flex 2 OCPU 12 GB	84.8.223.153	10.0.0.108	Worker Node + App Flask

Table 2.1 – Spécifications des machines virtuelles Oracle Cloud

Mise en place de l'infrastructure

3.1 Préparation de l'environnement local

3.1.1 Installation de Terraform

Terraform est installé sur Ubuntu 24.04 (WSL). Une attention particulière est portée au choix du dépôt : il convient d'utiliser `noble` (nom de code d'Ubuntu 24.04) et non `focal` (Ubuntu 20.04), sous peine d'obtenir une erreur `Unable to locate package terraform`.

```
hajar@LOTUS-FLOWER:~$ sudo apt-get update && sudo apt-get install -y gnupg software-properties-co
mmon
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 https://apt.releases.hashicorp.com noble InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Figure 3.1 – Installation des dépendances

```
hajar@LOTUS-FLOWER:~$ echo "deb [signed-by=/usr/share/keyrings/hashicorp.gpg] https://apt.release
s.hashicorp.com noble main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
deb [signed-by=/usr/share/keyrings/hashicorp.gpg] https://apt.releases.hashicorp.com noble main
```

Figure 3.2 – Ajout de dépôt Hashicorp en APT

```
hajar@LOTUS-FLOWER:~$ sudo apt-get update && sudo apt-get install -y terraform
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:5 https://apt.releases.hashicorp.com noble InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
```

Figure 3.3 – Installation de Terraform sur Ubuntu 24.04

```
hajar@LOTUS-FLOWER:~$ terraform --version
Terraform v1.14.6
on linux_amd64
```

Figure 3.4 – Vérifier la version de Terraform

3.2 Création manuelle des VMs sur Oracle Cloud

En raison des restrictions IAM du compte Free Tier, les instances VM ont été créées manuellement via la console Oracle Cloud. La procédure suivante a été appliquée pour chacune des deux machines (`k8s-master` et `k8s-worker`).

3.2.1 Informations de base et placement

Depuis la console : **Compute** → **Instances** → **Create Instance**. Le nom de l'instance (`k8s-master` ou `k8s-worker`) est renseigné, le compartiment sélectionné, et le domaine de disponibilité `AF-CASABLANCA-1-AD-1` choisi.

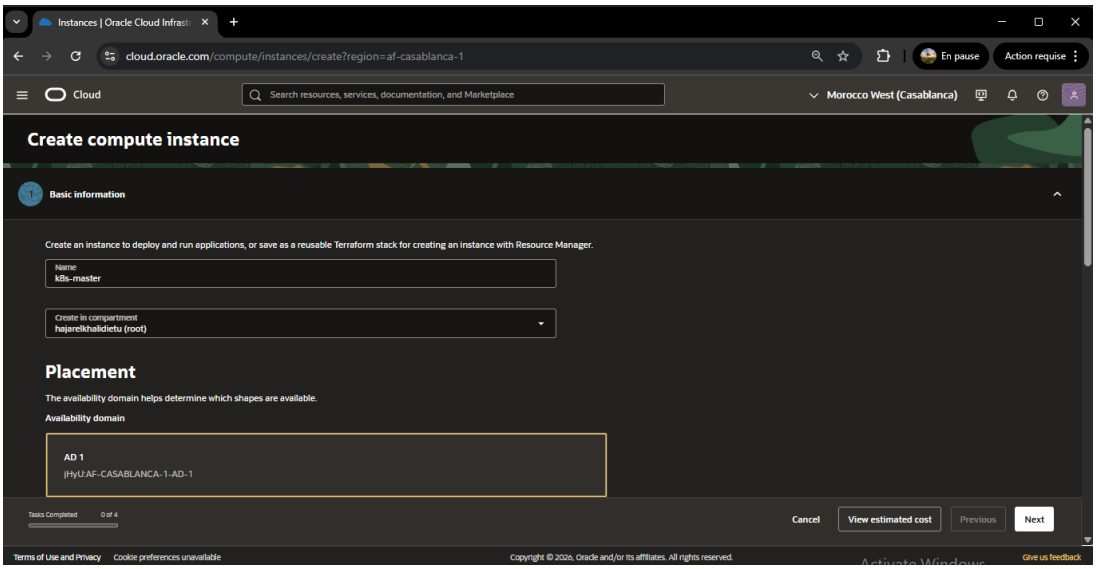

FIGURE 3.5 – Informations de base et placement de l'instance k8s-master

3.2.2 Choix de l'image et du shape

L'image sélectionnée est **Canonical Ubuntu 22.04** (build 2025.12.09-0). Le shape retenu est **VM.Standard.A1.Flex** (Always Free eligible), une machine virtuelle ARM Ampere avec 1 OCPU et 6 GB de mémoire, redimensionnée à 2 OCPUs et 12 GB lors de la configuration.

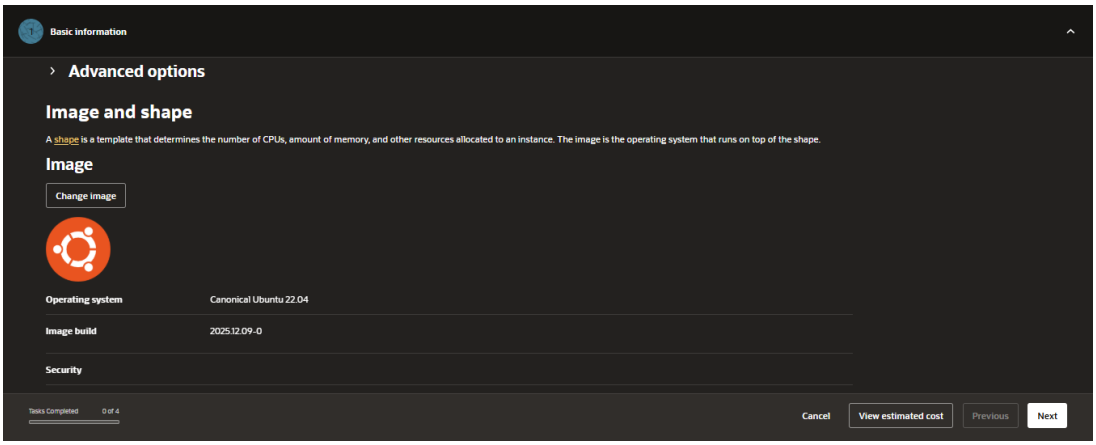

FIGURE 3.6 – Sélection de l'image Ubuntu 22.04

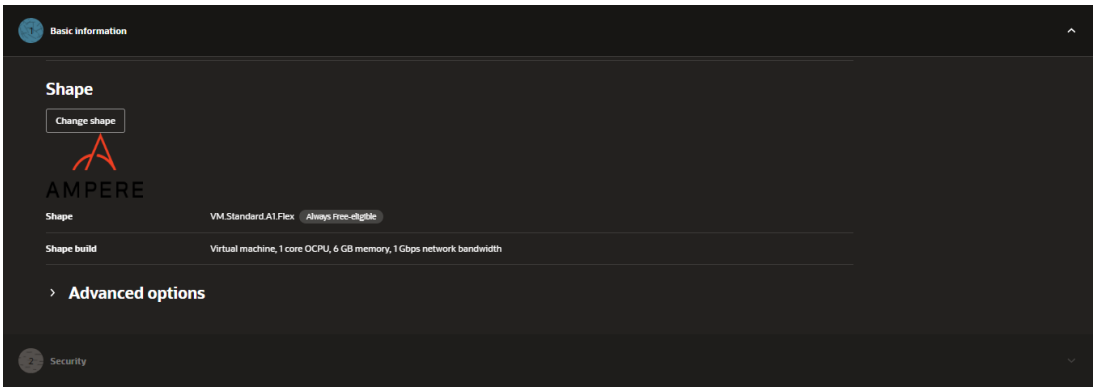

FIGURE 3.7 – Shape VM.Standard.A1.Flex Always Free

3.2.3 Configuration réseau

Dans la section **Networking**, le VCN `vcn-devops` est créé, ainsi que le subnet public associé. Une adresse IP publique est assignée automatiquement à chaque instance.

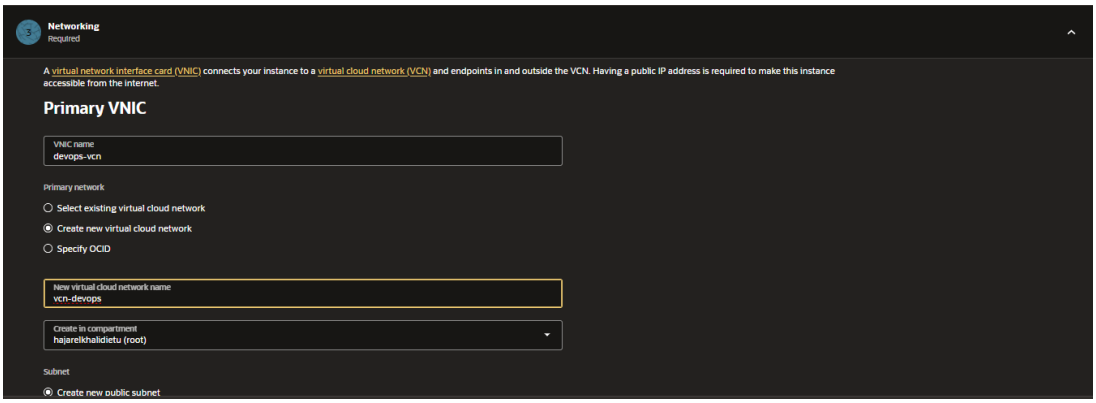

FIGURE 3.8 – Configuration réseau – sélection du VCN vcn-devops

3.2.4 Injection de la clé SSH

La clé publique SSH générée précédemment (`~/.ssh/id_rsa.pub`) est collée dans le champ **Paste public key**. Cette clé sera automatiquement injectée dans le fichier `~/.ssh/authorized_keys` de l'utilisateur `ubuntu` sur la VM, permettant la connexion SSH sans mot de passe.

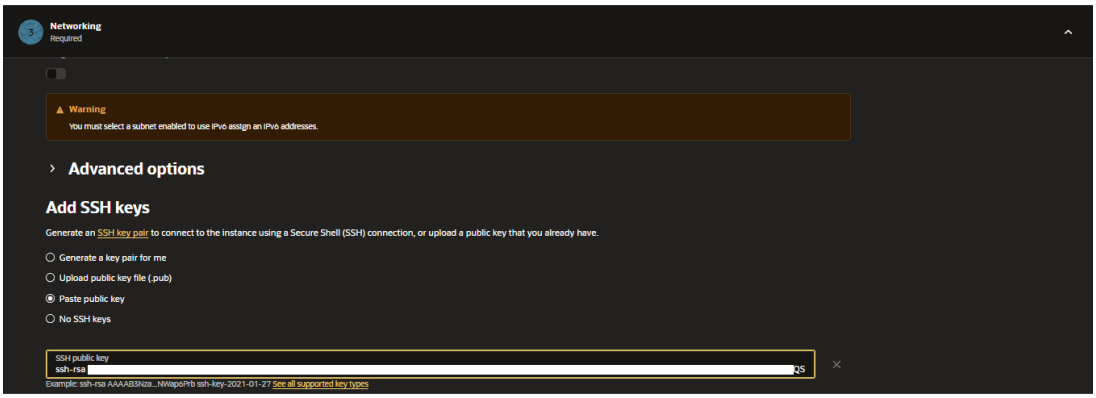

FIGURE 3.9 – Injection de la clé SSH publique

3.2.5 Récapitulatif et création

L'écran de révision affiche le résumé de la configuration avant validation : nom `k8s-master`, compartiment root, domaine AD-1, capacité on-demand. La même procédure est répétée pour créer `k8s-worker`.

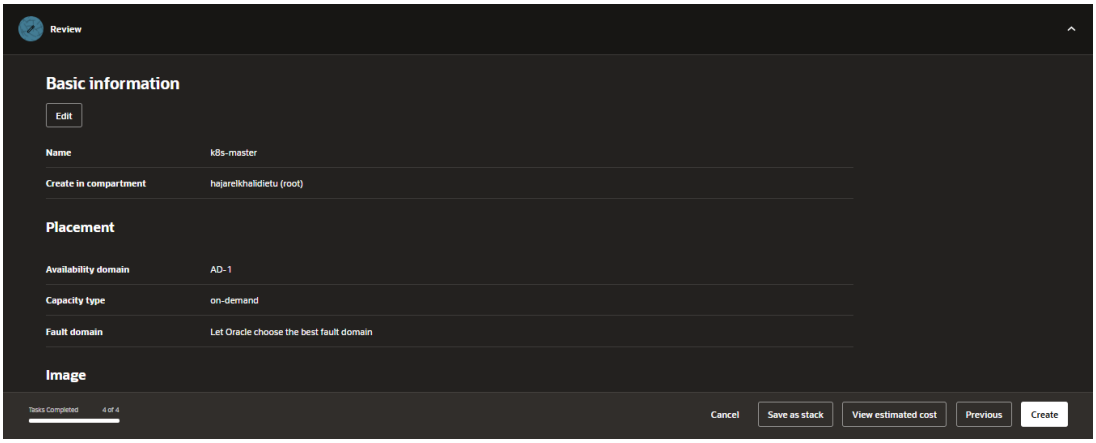

FIGURE 3.10 – Récapitulatif de la configuration avant création

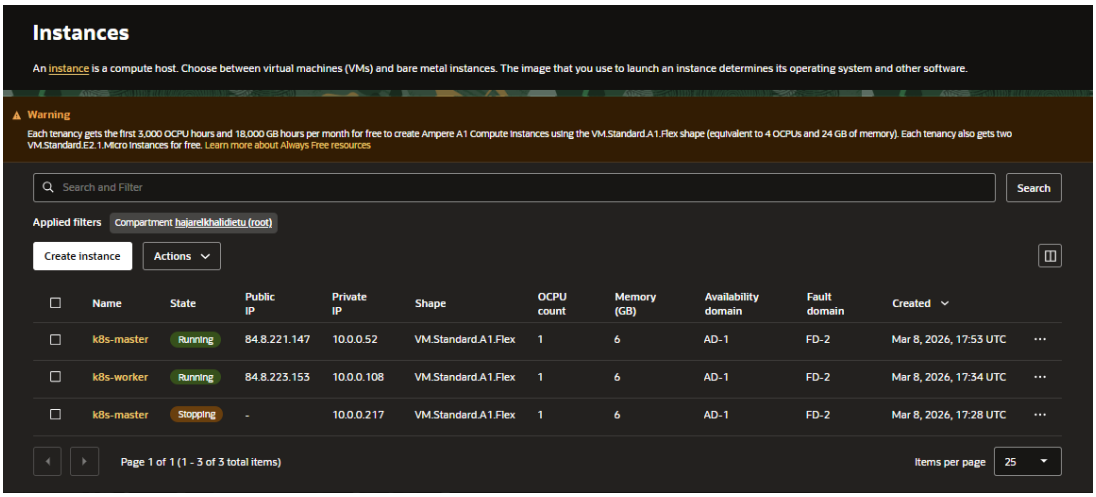

FIGURE 3.11 – Instances VMs créés

3.2.6 Configuration des accès Oracle Cloud

Pour permettre à Terraform de communiquer avec l'API Oracle Cloud, deux types de clés sont nécessaires. La **clé API OCI** authentifie les appels à l'API Oracle (Terraform, CLI). La **clé SSH** permet quant à elle de se connecter directement aux machines virtuelles.

La clé API est générée depuis la console Oracle Cloud (Profil → API Keys → Add API Key), puis placée dans `~/.oci/` avec les permissions appropriées. Le fichier de configuration `~/.oci/config` référence cette clé ainsi que les identifiants du tenancy (OCID utilisateur, OCID tenancy, fingerprint, région).

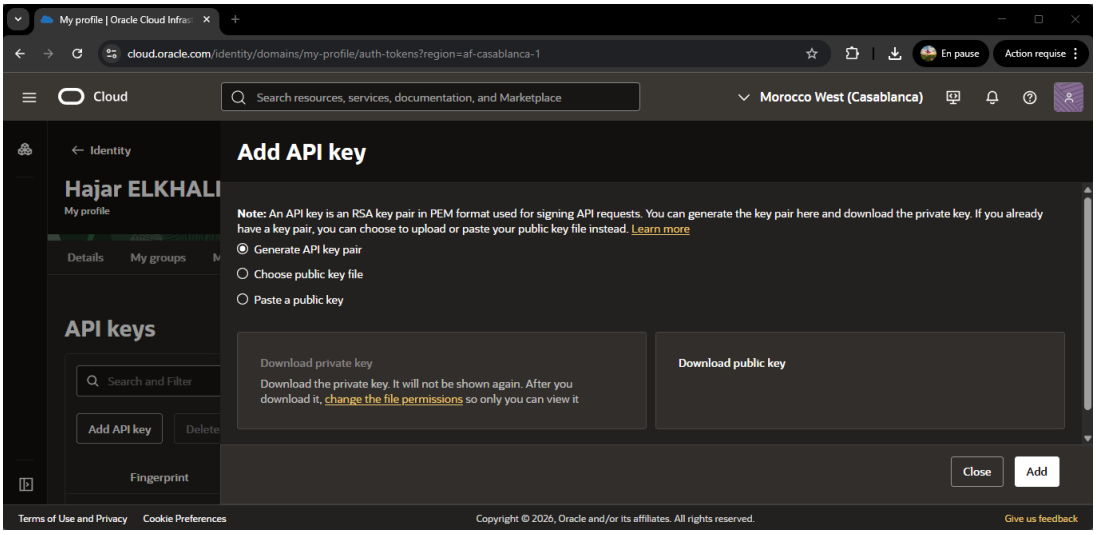

FIGURE 3.12 – Géneration de cle OCI

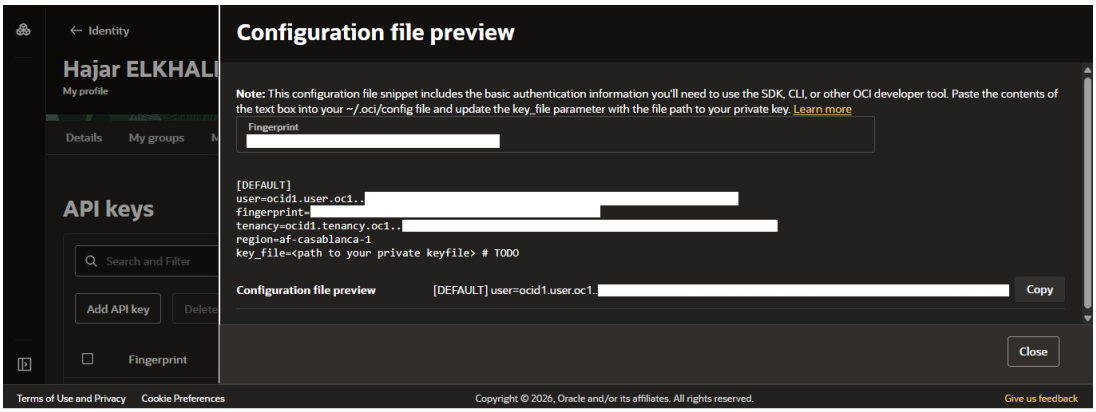

FIGURE 3.13 – Confirguration OCI file preview

3.2.7 Création du compartiment Oracle Cloud

Avant d'exécuter Terraform, il est nécessaire de créer un compartiment dédié au projet depuis la console Oracle Cloud. Un compartiment est une unité logique d'isolation des ressources dans Oracle Cloud. La procédure est la suivante : **C**onsole Oracle Cloud → menu hamburger → **Identity & Security** → **Compartments** → **Create Compartment**.

Une fois le compartiment créé, son OCID (identifiant unique) est récupéré depuis la page de détails et ajouté dans le fichier `main.tf` comme valeur de la variable `compartment_id`.

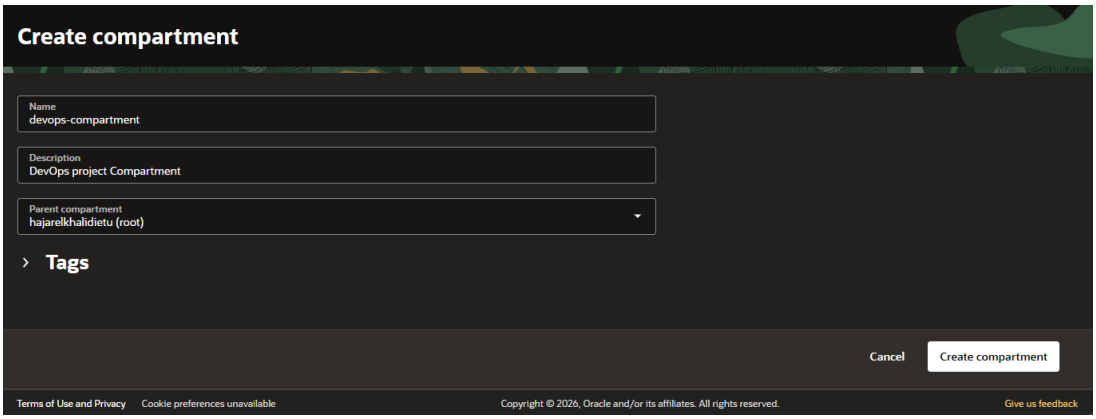

FIGURE 3.14 – Création de compartement

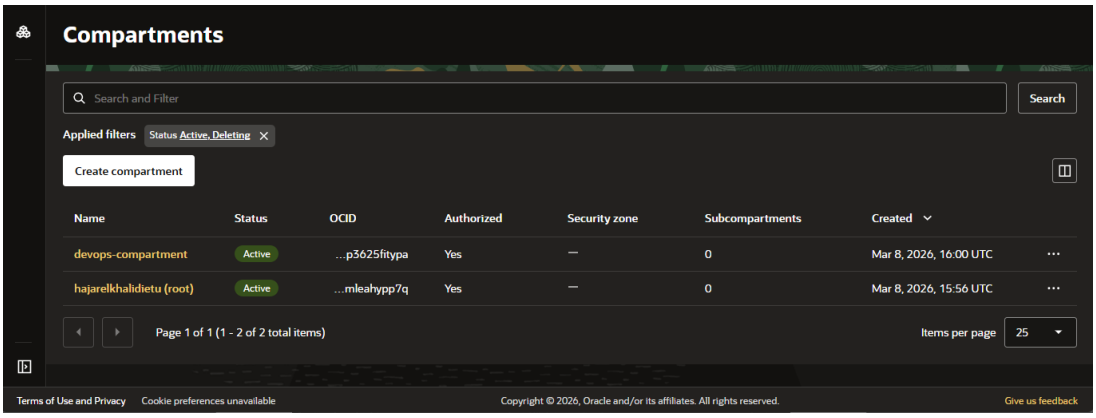

Figure 3.15 – Vérification de compartiment créé

3.2.8 Configuration des Policies IAM

Pour autoriser les opérations sur les ressources du compartiment, des policies IAM sont créées depuis : Console Oracle Cloud → menu hamburger → **Identity & Security** → **Policies** → **Create Policy**.

Les règles suivantes sont ajoutées via le Policy Builder pour permettre la gestion des instances, du réseau et des volumes dans le compartiment `devops` :

```
Allow any-user to manage instance-family in compartment devops
Allow any-user to manage virtual-network-family in compartment devops
Allow any-user to manage volume-family in compartment devops
Allow any-user to use images in compartment devops
```

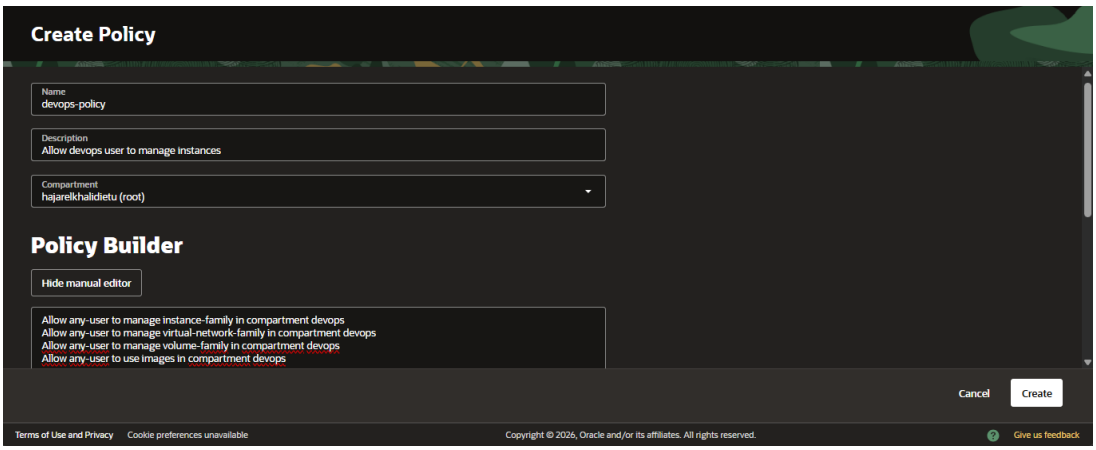

Figure 3.16 – Création des policies IAM pour le compartiment devops

3.2.9 Configuration des règles réseau (Security List)

Les règles d'entrée (Ingress Rules) de la Security List du VCN sont configurées pour autoriser les flux nécessaires au cluster Kubernetes et à l'application. Depuis la console : **Networking** → **Virtual Cloud Networks** → `vcn-devops` → **Security Lists**.

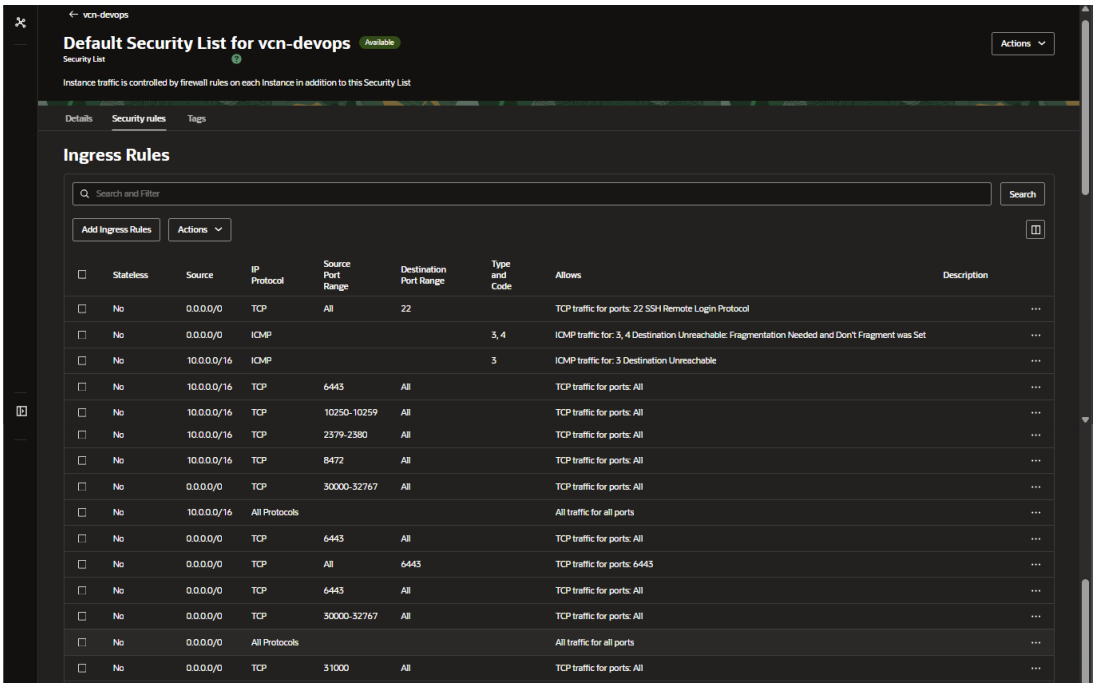

FIGURE 3.17 – Règles Ingress de la Security List du VCN devops

Listing 3.1 – Configuration OCI sur WSL en local

```
pip3 install oci-cli --break-system-packages
echo 'export PATH=$PATH:/home/hajar/.local/bin' >> ~/.bashrc
source ~/.bashrc
oci setup repair-file-permissions --file ~/.oci/config
oci iam availability-domain list --query "data[*].name" --output table
```

```
hajar@LOTUS-FLOWER:~$ oci iam availability-domain list --query "data[*].name" --output table
+-------------------------+
| Column1                 |
+-------------------------+
| jHyU:AF-CASABLANCA-1-AD-1 |
+-------------------------+
```

FIGURE 3.18 – Vérification de la configuration OCI

3.3 Provisionnement avec Terraform

3.3.1 Résolution du problème réseau

Avant d'initialiser Terraform, un problème de résolution DNS a été rencontré : WSL2 ne parvient pas à joindre `registry.terraform.io` via IPv6. La solution consiste à forcer la résolution vers l'adresse IPv4 en ajoutant une entrée dans `/etc/hosts` :

```
echo "13.224.83.61 registry.terraform.io" | sudo tee -a /etc/hosts
```

3.3.2 Description de l'infrastructure déclarée

Le fichier `terraform/main.tf` déclare l'ensemble des ressources réseau nécessaires au projet : un Virtual Cloud Network (VCN) avec le bloc CIDR `10.0.0.0/16`, un sous-réseau public (`10.0.1.0/24`), une passerelle Internet, les tables de routage associées, ainsi que les listes de sécurité contrôlant les flux entrants et sortants. Les deux instances VM sont également déclarées avec leur forme ARM (`VM.Standard.A1.Flex`), leur image Ubuntu 22.04, et la clé SSH publique injectée dans les métadonnées.

Listing 3.2 – Extrait de terraform/main.tf – Ressources réseau

```
resource "oci_core_vcn" "devops_vcn" {
  compartment_id = var.compartment_id
  cidr_blocks    = ["10.0.0.0/16"]
  display_name   = "devops-vcn"
}
resource "oci_core_subnet" "public_subnet" {
  compartment_id = var.compartment_id
  vcn_id         = oci_core_vcn.devops_vcn.id
  cidr_block     = "10.0.1.0/24"
  display_name   = "public-subnet"
}
resource "oci_core_instance" "master" {
  compartment_id      = var.compartment_id
  availability_domain = var.availability_domain
  shape               = "VM.Standard.A1.Flex"
  display_name        = "k8s-master"
  source_details {
    source_type = "image"
    source_id   = data.oci_core_images.ubuntu.images[0].id
  }
  create_vnic_details {
    subnet_id        = oci_core_subnet.public_subnet.id
    assign_public_ip = true
  }
```

```
  metadata = { ssh_authorized_keys = file("~/.ssh/id_rsa.pub") }
}
```

3.3.3 Exécution et résultats

L'exécution de Terraform suit le cycle classique `init` → `plan` → `apply`. Un problème de résolution DNS a nécessité l'ajout manuel de l'adresse IP de `registry.terraform.io` dans `/etc/hosts`.

Attention

Lors de l'`apply`, une erreur `404-NotAuthorizedOrNotFound` est apparue à la création des instances VM. Cause identifiée : les policies IAM du compte Free Tier root d'Oracle Cloud restreignent la création de ressources de calcul via l'API. Solution adoptée : les deux VMs ont été créées manuellement via la console Oracle Cloud, en réutilisant le VCN et le subnet provisionnés par Terraform.

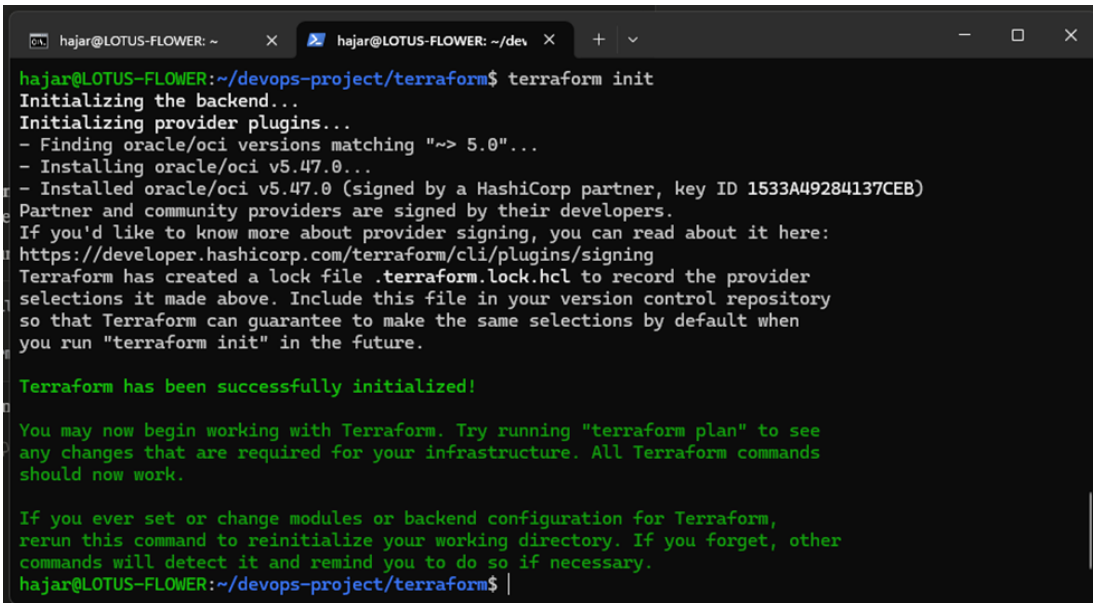

Figure 3.19 – Vérification de la configuration OCI

3.4 Configuration avec Ansible

3.4.1 Inventaire et connectivité

L'inventaire Ansible définit les deux machines cibles, leurs adresses IP publiques, le compte utilisateur (`ubuntu`) et le chemin vers la clé SSH privée. Un test de connectivité via `ansible all -m ping` précède systématiquement l'exécution du playbook.

Listing 3.3 – ansible/inventory.ini

```
[master]
k8s-master ansible_host=84.8.221.147 ansible_user=ubuntu \
  ansible_ssh_private_key_file=~/.ssh/id_rsa

[worker]
k8s-worker ansible_host=84.8.223.153 ansible_user=ubuntu \
  ansible_ssh_private_key_file=~/.ssh/id_rsa

[all:vars]
ansible_python_interpreter=/usr/bin/python3
```

3.4.2 Playbook de configuration

Le playbook `playbook-setup.yml` automatise l'installation complète de Docker et Kubernetes sur les deux nœuds. Plusieurs points méritent une attention particulière. D'abord, l'architecture ARM64 des VMs Oracle A1.Flex impose d'utiliser `arch=arm64` dans les sources APT de Docker, et le dépôt `jammy` (Ubuntu 22.04) pour la compatibilité. Ensuite, `containerd` doit être reconfiguré avec `SystemdCgroup = true` pour éviter des conflits avec `kubelet`. Enfin, les modules noyau `overlay` et `br_netfilter` ainsi que les paramètres sysctl associés doivent être activés pour le bon fonctionnement du réseau Kubernetes.

Listing 3.4 – Extrait du playbook – Configuration de containerd

```
- name: Configure containerd for Kubernetes
  shell: |
    mkdir -p /etc/containerd
    containerd config default > /etc/containerd/config.toml
    sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' \
      /etc/containerd/config.toml

- name: Load kernel modules
  modprobe:
    name: "{{ item }}"
  loop: [overlay, br_netfilter]

- name: Set sysctl params for Kubernetes
  sysctl:
    name: "{{ item.key }}"
    value: "{{ item.value }}"
    reload: yes
  loop:
    - { key: 'net.bridge.bridge-nf-call-iptables', value: '1' }
    - { key: 'net.ipv4.ip_forward', value: '1' }
```

3.4.3 Exécution du playbook

Avant de lancer le playbook, la connectivité SSH vers les deux machines est vérifiée :

```
hajar@LOTUS-FLOWER:~/devops-project/ansible$ ansible all -i inventory.ini -m ping
k8s-worker | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
k8s-master | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

FIGURE 3.20 – Vérification de la connectivité SSH vers les deux noeuds

Le playbook est ensuite exécuté sur l'ensemble des noeuds :

```
hajar@LOTUS-FLOWER: ~   hajar@LOTUS-FLOWER: ~/dev   hajar@LOTUS-FLOWER: ~

hajar@LOTUS-FLOWER:~/devops-project/ansible$ ansible-playbook -i inventory.ini playbook-setup.yml

PLAY [Install Docker and Kubernetes on all nodes] ******************************************************************************

TASK [Gathering Facts] *********************************************************************************************************
ok: [k8s-worker]
ok: [k8s-master]

TASK [Update apt packages] *****************************************************************************************************
changed: [k8s-master]
changed: [k8s-worker]

TASK [Install prerequisites] ***************************************************************************************************
ok: [k8s-master]
ok: [k8s-worker]

TASK [Add Docker GPG key] ******************************************************************************************************
ok: [k8s-worker]
changed: [k8s-master]

TASK [Add Docker repository (ARM64)] *******************************************************************************************
ok: [k8s-worker]
changed: [k8s-master]

TASK [Install Docker] **********************************************************************************************************
ok: [k8s-worker]
changed: [k8s-master]

TASK [Enable Docker service] ***************************************************************************************************
ok: [k8s-master]
ok: [k8s-worker]

TASK [Add ubuntu user to docker group] *****************************************************************************************
ok: [k8s-worker]
changed: [k8s-master]

TASK [Add Kubernetes GPG key] **************************************************************************************************
ok: [k8s-worker]
changed: [k8s-master]

TASK [Add Kubernetes repository] ***********************************************************************************************
ok: [k8s-worker]
changed: [k8s-master]

TASK [Install kubeadm, kubelet, kubectl] ***************************************************************************************
ok: [k8s-worker]
changed: [k8s-master]

TASK [Hold Kubernetes versions] ************************************************************************************************
changed: [k8s-master] => (item=kubelet)
ok: [k8s-worker] => (item=kubelet)
changed: [k8s-master] => (item=kubeadm)
ok: [k8s-worker] => (item=kubeadm)
changed: [k8s-master] => (item=kubectl)
ok: [k8s-worker] => (item=kubectl)

TASK [Disable swap] ************************************************************************************************************
changed: [k8s-worker]
changed: [k8s-master]

TASK [Remove swap from fstab] **************************************************************************************************
ok: [k8s-worker]
ok: [k8s-master]

TASK [Enable kernel modules for Kubernetes] ************************************************************************************
ok: [k8s-worker] => (item=overlay)
ok: [k8s-master] => (item=overlay)
ok: [k8s-worker] => (item=br_netfilter)
changed: [k8s-master] => (item=br_netfilter)

TASK [Set sysctl params for Kubernetes] ****************************************************************************************
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
[DEPRECATION WARNING]: Importing 'to_native' from 'ansible.module_utils._text' is deprecated. This feature will be removed from ansible-core version
 2.24. Use ansible.module_utils.common.text.converters instead.
changed: [k8s-master] => (item={'key': 'net.bridge.bridge-nf-call-iptables', 'value': '1'})
ok: [k8s-worker] => (item={'key': 'net.bridge.bridge-nf-call-iptables', 'value': '1'})
changed: [k8s-master] => (item={'key': 'net.bridge.bridge-nf-call-ip6tables', 'value': '1'})
ok: [k8s-worker] => (item={'key': 'net.bridge.bridge-nf-call-ip6tables', 'value': '1'})
changed: [k8s-master] => (item={'key': 'net.ipv4.ip_forward', 'value': '1'})
ok: [k8s-worker] => (item={'key': 'net.ipv4.ip_forward', 'value': '1'})

PLAY RECAP *********************************************************************************************************************
k8s-master                 : ok=16   changed=12   unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
k8s-worker                 : ok=16   changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

hajar@LOTUS-FLOWER:~/devops-project/ansible$
```

FIGURE 3.21 – Exécution du playbook Ansible sur les noeuds master et worker

3.5 Déploiement du cluster Kubernetes

3.5.1 Initialisation du nœud master

Le cluster Kubernetes est initialisé via `kubeadm init` sur le nœud master. Le paramètre `-pod-network-cidr=10.244.0.0/16` correspond au réseau attendu par Flannel, le plugin réseau (CNI) choisi pour sa simplicité de déploiement. L'adresse IP privée du master (`10.0.0.52`) est spécifiée comme adresse de l'API Server.

Listing 3.5 – Initialisation du cluster sur le nœud master

```
sudo kubeadm init \
  --pod-network-cidr=10.244.0.0/16 \
  --apiserver-advertise-address=10.0.0.52

mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Installer Flannel
kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/
   master/Documentation/kube-flannel.yml
```

```
ubuntu@vcn-devops:~$ kubectl apply -f https://raw.githubusercontent.com/
flannel-io/flannel/master/Documentation/kube-flannel.yml
namespace/kube-flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
serviceaccount/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
```

Figure 3.22 – Installation du plugin réseau Flannel sur le cluster

La même configuration requis sur Ubuntu 22.04 de `containerd` appliquée sur le master est reproduite sur le worker avant de le joindre au cluster :

Listing 3.6 – Configuration de containerd appliquée sur les noeuds worker et master

```
sudo mkdir -p /etc/containerd
sudo containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' \
  /etc/containerd/config.toml
sudo systemctl restart containerd
```

3.5.2 Jonction du nœud worker

La commande de jonction est générée automatiquement par `kubeadm token create -print-join-command` sur le master, puis exécutée sur le worker après la même configuration de containerd.

```
hajar@LOTUS-FLOWER: ~   hajar@LOTUS-FLOWER: ~/devo   ubuntu@devops-vcn-571552:   ubuntu@vcn-devops: ~

 rotatable kubelet client certificate and key
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular
user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed
at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on
 each as root:

kubeadm join 10.0.0.52:6443 --token [redacted] \
        --discovery-token-ca-cert-hash sha256:49ecf16e0d725bfcf61800a58e
[redacted]

ubuntu@devops-vcn-571552:~$ sudo kubeadm join 10.0.0.52:6443 --token [redacted]
[redacted] \
        --discovery-token-ca-cert-hash sha256:49ecf16e0d725bfcf61800a58e
[redacted]
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-
system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/
config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/va
r/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was r
eceived.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the c
luster.

ubuntu@devops-vcn-571552:~$
```

Figure 3.23 – Initialisation du master et jonction du worker au cluster

3.5.3 Vérification de l'état du cluster

Résultat

NAME	STATUS	ROLES	AGE	VERSION
devops-vcn-571552	Ready	<none>	6m19s	v1.28.15
vcn-devops	Ready	control-plane	98m	v1.28.15

Les deux nœuds sont en statut **Ready** : le cluster Kubernetes est opérationnel.

```
ubuntu@vcn-devops:~$ kubectl get nodes
NAME                STATUS   ROLES           AGE     VERSION
devops-vcn-571552   Ready    <none>          6m19s   v1.28.15
vcn-devops          Ready    control-plane   98m     v1.28.15
```

Figure 3.24 – État du cluster Kubernetes après initialisation

3.5.4 Problèmes rencontrés et solutions

Plusieurs obstacles techniques ont été rencontrés lors du déploiement du cluster, documentés dans le tableau suivant.

Erreur	**Cause**	**Solution**
NumCPU error	VMs initiales avec 1 OCPU seulement	Redimensionnement à 2 OCPUs par VM
br_netfilter manquant	Module noyau non chargé au démarrage	Ajout de `modprobe` et configuration sysctl
Port 6443 inaccessible	Pare-feu Oracle Cloud strict	Ajout d'une règle Ingress TCP 6443
Erreur TLS certificat	IP publique absente du certificat kubeadm	Utilisation de `insecure-skip-tls-verify`
exec format error	Image Docker x86 sur ARM64	Build multi-arch linux/amd64+linux/arm64

Table 3.1 – Problèmes rencontrés lors du déploiement Kubernetes

Application, CI/CD et Monitoring

4.1 Application Flask conteneurisée

4.1.1 Code source

L'application est une API REST minimaliste développée en Python avec le framework Flask. Elle expose deux routes : la route principale `/` qui retourne des informations générales sur le service (message, version, statut), et la route `/health` utilisée par les sondes de vivacité et de disponibilité de Kubernetes pour déterminer si le conteneur est sain.

Listing 4.1 – app/app.py – API REST Flask

```
from flask import Flask, jsonify
import os

app = Flask(__name__)

@app.route('/')
def home():
    return jsonify({
        'message': 'API DevOps - Master DSBD & IA',
        'version': os.getenv('APP_VERSION', '1.0.0'),
        'status': 'running'
    })

@app.route('/health')
def health():
    return jsonify({'status': 'healthy'}), 200

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

4.1.2 Dockerfile et build multi-architecture

Le Dockerfile utilise l'image de base `python:3.11-slim` et lance l'application via Gunicorn, un serveur WSGI adapté à la production. Le point critique est la nécessité de construire l'image pour deux architectures simultanément, en raison de l'incompatibilité entre les runners GitHub Actions (x86/amd64) et les VMs Oracle (ARM64).

Listing 4.2 – app/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
ENV APP_VERSION=1.0.0
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

4.1.3 Tests unitaires

Deux tests pytest valident le comportement de l'application : `test_home` vérifie que la route `/` retourne un code HTTP 200 et un champ `status` égal à `running`, et `test_health` vérifie que `/health` retourne `healthy`. Ces tests sont exécutés automatiquement par le pipeline CI/CD à chaque push.

Listing 4.3 – app/tests/test_app.py – Tests unitaires pytest

```
import pytest
import sys
import os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..'))
from app import app

@pytest.fixture
def client():
    app.config['TESTING'] = True
    with app.test_client() as client:
        yield client

def test_home(client):
    response = client.get('/')
    assert response.status_code == 200
    data = response.get_json()
    assert 'message' in data
    assert data['status'] == 'running'

def test_health(client):
```

```
    response = client.get('/health')
    assert response.status_code == 200
    data = response.get_json()
    assert data['status'] == 'healthy'
```

4.2 Pipeline CI/CD avec GitHub Actions

4.2.1 Secrets et prérequis

Avant la mise en place du pipeline, quatre secrets sont configurés dans les paramètres du dépôt GitHub : `DOCKERHUB_USERNAME` et `DOCKERHUB_TOKEN` pour la publication de l'image, `KUBE_CONFIG` contenant le fichier `kubeconfig` du cluster encodé en base64, et `MASTER_SSH_KEY` pour les éventuelles connexions SSH depuis le pipeline.

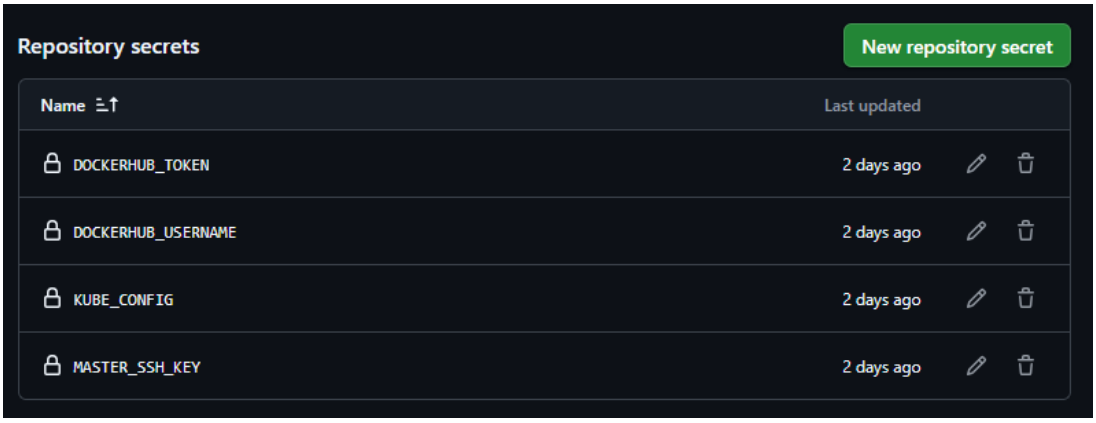

Figure 4.1 – Secrets configurés dans le dépôt GitHub Actions

4.2.2 Structure du pipeline

Le pipeline est défini dans `.github/workflows/ci-cd.yml` et se décompose en trois jobs séquentiels exécutés à chaque push sur la branche `main`.

Le premier job (**test**) installe les dépendances Python et exécute les tests pytest. En cas d'échec, le pipeline s'arrête immédiatement, empêchant le déploiement d'une version défectueuse.

Le deuxième job (**build-push**) utilise QEMU et Docker Buildx pour construire l'image en multi-architecture (`linux/amd64` et `linux/arm64`) et la pousser sur Docker Hub avec deux tags : le tag `latest` et le SHA du commit pour la traçabilité.

Le troisième job (**deploy**) configure `kubectl` avec le kubeconfig du cluster, puis met à jour l'image du Deployment Kubernetes et attend la fin du rollout.

Listing 4.4 – Extrait du workflow – Job build-push multi-architecture

```
build-push:
  name: Build et Push Docker (Multi-arch)
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: docker/setup-qemu-action@v3
    - uses: docker/setup-buildx-action@v3
    - uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKERHUB_USERNAME }}
        password: ${{ secrets.DOCKERHUB_TOKEN }}
    - uses: docker/build-push-action@v5
      with:
        context: ./app
        platforms: linux/amd64,linux/arm64
        push: true
        tags: |
          ${{ env.IMAGE_NAME }}:latest
          ${{ env.IMAGE_NAME }}:${{ github.sha }}
```

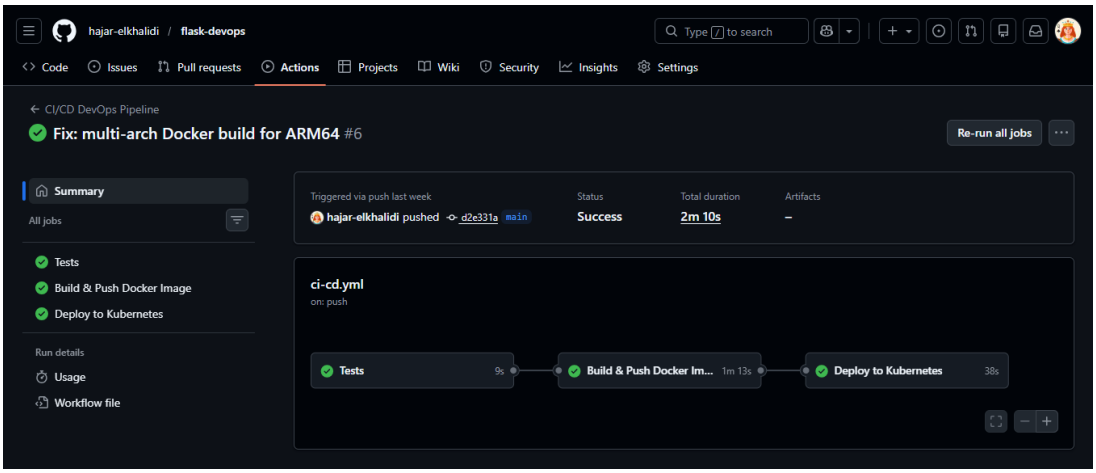

FIGURE 4.2 – Pipeline CI/CD GitHub Actions exécuté avec succès

4.3 Déploiement Kubernetes de l'application

4.3.1 Manifest Deployment

Le Deployment Kubernetes définit deux réplicas de l'application Flask, garantissant la haute disponibilité. Des sondes de vivacité (`livenessProbe`) et de disponibilité (`readinessProbe`) interrogent périodiquement la route `/health` pour détecter les conteneurs défaillants et les

exclure du routage. Des limites de ressources sont également imposées (64–128 Mi RAM, 100–200 m CPU) pour protéger les autres charges de travail du cluster.

```
ubuntu@vcn-devops:~$ kubectl get pods -n monitoring
NAME                                                     READY   STATUS    RESTARTS   AGE
alertmanager-kube-prometheus-kube-prome-alertmanager-0   2/2     Running   0          42h
kube-prometheus-grafana-cd644499f-kbvsl                  3/3     Running   0          42h
kube-prometheus-kube-prome-operator-77bdb457f9-k8s52     1/1     Running   0          42h
kube-prometheus-kube-state-metrics-79cf56b7f9-tpvgq      1/1     Running   0          42h
kube-prometheus-prometheus-node-exporter-9xttf           1/1     Running   0          42h
kube-prometheus-prometheus-node-exporter-kz4f2           1/1     Running   0          42h
prometheus-kube-prometheus-kube-prome-prometheus-0       2/2     Running   0          42h
ubuntu@vcn-devops:~$ client_loop: send disconnect: Broken pipe
hajar@LOTUS-FLOWER:~$
```

FIGURE 4.3 – Pods de l'application Flask en statut Running

4.3.2 Service NodePort

Le Service de type `NodePort` expose l'application sur le port 30000 de chaque nœud du cluster, permettant l'accès depuis l'extérieur via l'IP publique du worker (`http://84.8.223.153:30000`).

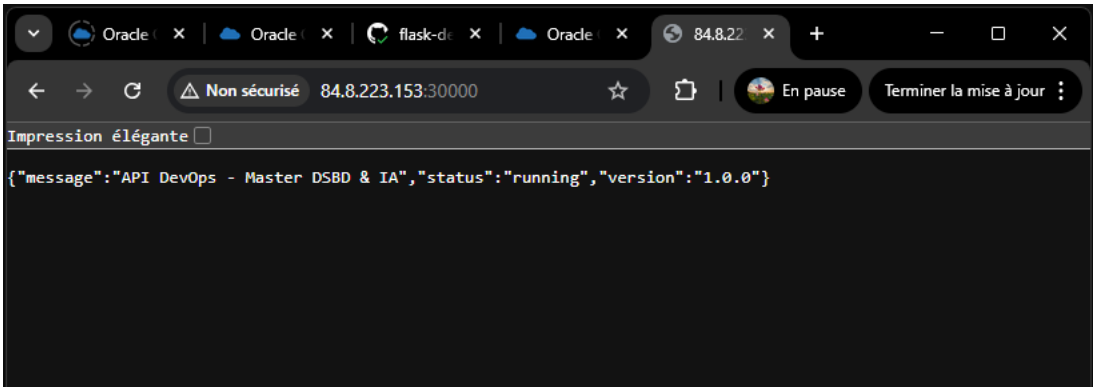

FIGURE 4.4 – Application Flask accessible dans le navigateur]

4.3.3 Interface utilisateur

Afin de visualiser les données retournées par l'API de manière conviviale, une interface web légère a été développée en HTML/CSS/JavaScript et intégrée directement dans l'application Flask. Elle est accessible via la route `/ui` et se connecte automatiquement aux endpoints `/` et `/health` pour afficher en temps réel le statut de l'API, la version déployée et le résultat du health check. L'interface se rafraîchit automatiquement toutes les 30 secondes.

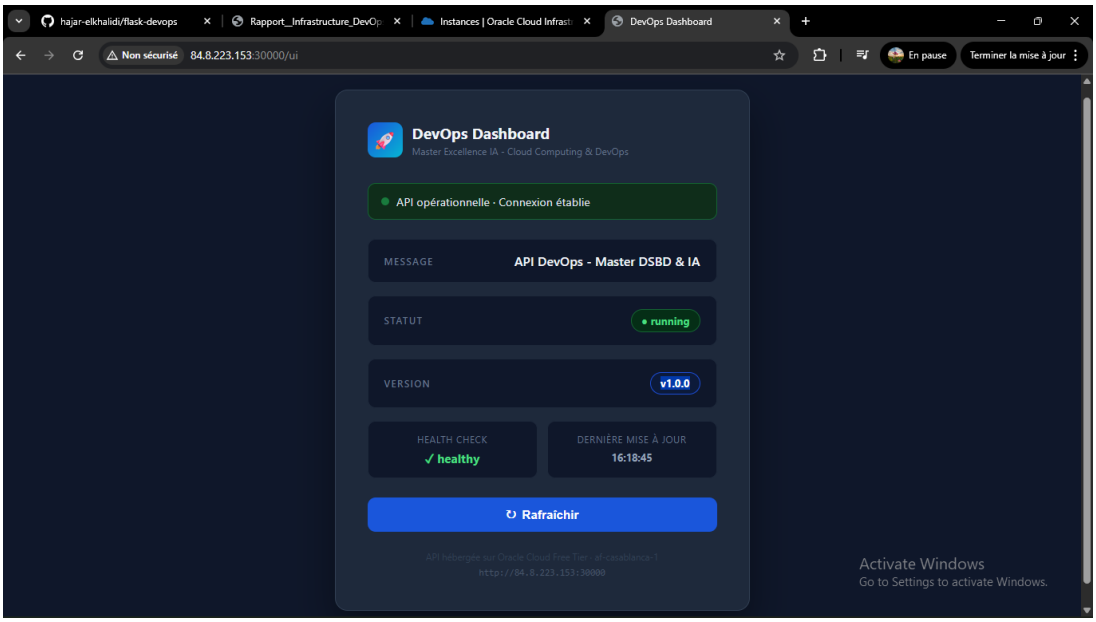

FIGURE 4.5 – Interface web de supervision de l'API Flask

4.4 Monitoring avec Prometheus et Grafana

4.4.1 Déploiement via Helm

Le stack de monitoring est déployé dans un namespace dédié `monitoring` via le chart Helm `kube-prometheus-stack`. Ce chart embarque Prometheus, Grafana, Alertmanager, Node Exporter et kube-state-metrics. Grafana est exposé via un Service NodePort sur le port 31000.

Listing 4.5 – Installation du stack de monitoring

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update
kubectl create namespace monitoring
helm install kube-prometheus prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set grafana.service.type=NodePort \
  --set grafana.service.nodePort=31000
```

4.4.2 Composants déployés

Composant	Port	Rôle
Grafana	31000 (NodePort)	Visualisation des métriques et dashboards
Prometheus	9090 (ClusterIP)	Collecte et stockage des métriques
Alertmanager	9093 (ClusterIP)	Gestion et routage des alertes
Node Exporter	9100 (ClusterIP)	Métriques système des nœuds
kube-state-metrics	8080 (ClusterIP)	Métriques des ressources Kubernetes

Table 4.1 – Composants du stack de monitoring déployés dans le namespace monitoring

4.4.3 Configuration et dashboards

Après connexion à l'interface Grafana (`http://84.8.223.153:31000`), Prometheus est ajouté comme source de données via son adresse ClusterIP interne. Trois dashboards communautaires sont importés pour visualiser l'état du cluster : le dashboard **6417** (Kubernetes Cluster), le **1860** (Node Exporter Full) pour les métriques système des nœuds, et le **15760** pour une vue détaillée de la consommation de ressources.

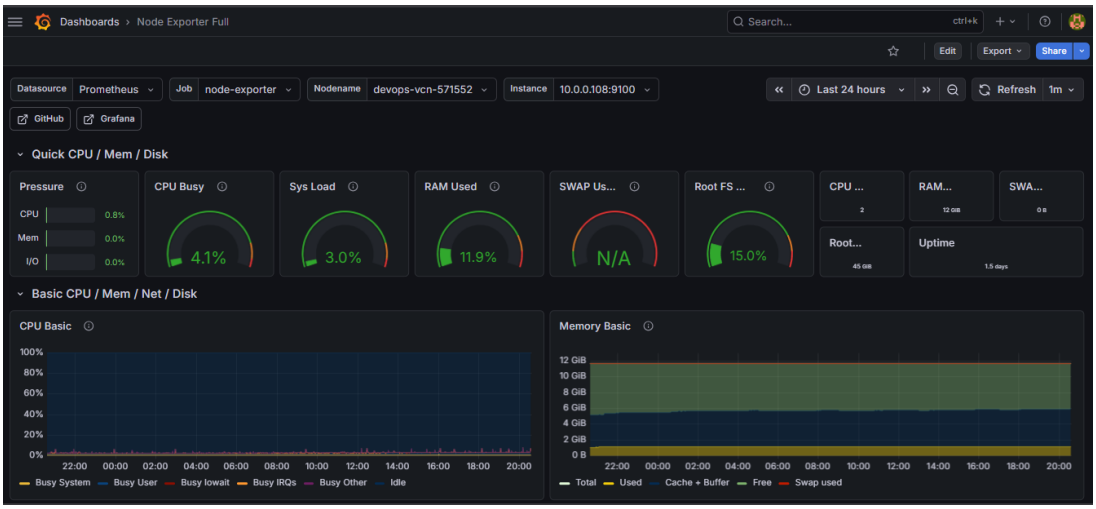

Figure 4.6 – Dashboard Grafana de supervision du cluster Kubernetes]

Bilan et conclusion

5.1 Synthèse des réalisations

L'ensemble des objectifs fixés en début de projet ont été atteints. Le tableau suivant récapitule les livrables produits et leur statut.

Livrable	Statut	Détails
Application Flask	Réalisé	API REST avec routes `/` et `/health`
Dockerfile	Réalisé	Image multi-arch linux/amd64 + linux/arm64
Tests pytest	Réalisé	2 tests unitaires valident les routes
Terraform	Réalisé	VCN, subnet, et règles réseau sur Oracle Cloud
Ansible	Réalisé	Docker, Kubernetes, containerd configurés
Cluster Kubernetes	Réalisé	2 nœuds Ready, version v1.28.15
Pipeline CI/CD	Réalisé	Test, build multi-arch, deploy automatisés
Docker Hub	Réalisé	Image `hajare/flask-devops` publiée
Manifests Kubernetes	Réalisé	Deployment 2 réplicas + Service NodePort
Monitoring	Réalisé	Prometheus + Grafana opérationnels
Documentation	Réalisé	Rapport détaillé reproductible

Table 5.1 – Récapitulatif des livrables du projet

5.2 Difficultés rencontrées

La réalisation de ce projet a nécessité de surmonter plusieurs obstacles techniques significatifs.

La principale difficulté a été liée aux **restrictions IAM d'Oracle Cloud Free Tier**. Bien que Terraform ait correctement provisionné le réseau, la création des instances VM via l'API a été systématiquement rejetée avec une erreur `404-NotAuthorizedOrNotFound`. La cause, identifiée après investigation, réside dans les policies par défaut du compte root Free Tier qui limitent certaines opérations via API. La solution retenue – création manuelle des VMs via la console – illustre l'importance de maîtriser les limites de l'automatisation.

La **compatibilité ARM64** a été un défi transversal. L'architecture ARM des VMs Oracle A1.Flex est incompatible avec les images Docker standard (amd64), et les runners GitHub Actions sont exclusivement x86. Cette contrainte a imposé la mise en place d'un build multi-

architecture via QEMU et Docker Buildx, une technique qui allonge significativement le temps de build (10–15 minutes contre 2–3 minutes pour une architecture unique).

La **sécurisation des communications** entre GitHub Actions et le cluster Kubernetes a également demandé plusieurs itérations. Le certificat généré par kubeadm couvrant uniquement les adresses internes, il a fallu utiliser l'option `insecure-skip-tls-verify` pour permettre les connexions via l'IP publique. Une alternative plus sécurisée consisterait à regénérer le certificat en y incluant l'IP publique via `kubeadm init -apiserver-cert-extra-sans`.

5.3 Perspectives

Plusieurs améliorations sont envisageables pour ce projet. Sur le plan de la sécurité, le remplacement de `insecure-skip-tls-verify` par un certificat correctement signé constituerait une première priorité. L'ajout d'un gestionnaire de secrets type Vault permettrait de mieux protéger les credentials appliqués dans le pipeline.

Sur le plan de la résilience, l'ajout d'un troisième nœud worker et la configuration de l'autoscaling horizontal (`HorizontalPodAutoscaler`) amélioreraient la disponibilité de l'application. La mise en place d'un Ingress Controller avec un nom de domaine et un certificat TLS remplacerait avantageusement le Service NodePort actuel.

Enfin, l'intégration d'Alertmanager avec des notifications (e-mail, Slack) compléterait la chaîne de supervision et permettrait une réactivité accrue face aux incidents de production.

5.4 Conclusion

Ce projet a permis de concevoir et de déployer une infrastructure DevOps complète sur Oracle Cloud Free Tier, depuis le provisionnement des machines virtuelles jusqu'au monitoring en production. L'ensemble des pratiques modernes du domaine ont été mobilisées : Infrastructure as Code, conteneurisation multi-architecture, orchestration Kubernetes, intégration et déploiement continus, et supervision.

Au-delà des aspects techniques, ce projet a démontré l'importance d'une approche méthodique face aux obstacles : chaque erreur rencontrée a été analysée, documentée et résolue de manière raisonnée. La documentation produite constitue un guide reproductible pour déployer une infrastructure similaire dans un contexte académique ou professionnel.